

Tema 1 – Princípios de Segurança em DevOps

A integração da segurança no DevOps visa criar sistemas robustos através da automação e de uma cultura organizacional centrada na proteção dos dados e infraestrutura.

- **Integração e Entrega Contínua (CI/CD):** A entrega contínua fragmenta o software em partes menores lançadas regularmente, permitindo alterações ágeis e seguras com base na experiência do usuário. Diferente do modelo tradicional, o software pode ser publicado a qualquer momento.
- **Ciclo de Vida Seguro:** Práticas de segurança são incorporadas desde a definição de requisitos até o teste e implantação, garantindo a proteção da infraestrutura em nuvem.
- **Monitoramento Contínuo:** Essencial para identificar atividades suspeitas e anomalias em tempo real através da análise de logs e métricas.
- **Gestão de Identidade e Acesso (IAM):** Foca no princípio do menor privilégio, autenticação multifatorial e monitoramento rigoroso.
- **Resposta a Incidentes:** Estabelecimento de protocolos para ações rápidas e comunicação transparente em caso de violações.
- **Testes Automatizados:** No DevOps, a execução manual de testes é inviável devido aos ciclos curtos. Testes automatizados reduzem o tempo de entrega e o risco de falhas.
- **Testes com Terraform:** Como ferramenta de Infraestrutura como Código (IaC), o Terraform permite testes de unidade, integração (comunicação entre partes), aceitação (atendimento aos requisitos do usuário) e integração contínua (validação de sintaxe e plano). O ciclo de vida no Terraform envolve: Planejamento, Aplicação, Monitoramento, Mudanças e Destruição.

Tema 2 – Ferramentas para Análise e Gestão de Riscos

O uso de ferramentas especializadas, como o Terraform, permite a identificação e mitigação proativa de ameaças.

- **Processo de Análise de Riscos:** Envolve a identificação (geração de lista de recursos), avaliação (vulnerabilidades e impacto) e mitigação (controles de segurança e backups).
- **Gestão de Riscos:** Foca no monitoramento dos riscos identificados, gerenciamento de mudanças para evitar novos riscos e resposta a incidentes (recuperação de desastres).
- **Tipos de Riscos Analisados:** Incluem riscos de segurança (acessos não autorizados), disponibilidade (interrupções), desempenho (latência) e custo (recursos ociosos).
- **Vantagens e Considerações:** A automatização traz precisão e visibilidade. É necessário considerar a competência técnica da equipe, escalabilidade e custos de implementação.
- **Exemplos Práticos na AWS:** O uso do Terraform deve mitigar riscos como exposição de credenciais (usando variáveis seguras), configurações incorretas de grupos de segurança e inconsistências no ambiente (validando com `terraform plan`).

Tema 3 – Confiabilidade e Continuidade em DevOps

Busca garantir a estabilidade e a entrega consistente de valor por meio de resiliência e feedback rápido.

- **Contornando Problemas (IaC):** O aumento de recursos pode gerar "desvios de configuração". Servidores modificados manualmente fora da automação tornam-se "flocos de neve", tornando a infraestrutura frágil e irreplicável.
- **O Ciclo do Medo:** Alterações manuais geram servidores inconsistentes, o que causa medo de que a automação quebre o sistema, retroalimentando o ciclo de mudanças manuais.
- **6 Passos para a Implementação do DevOps:**
 - **Definição do projeto ideal:** Começar com projetos de baixo risco (Web/Mobile).
 - **Constituição do time:** Equipe multidisciplinar e aberta a mudanças.
 - **Adoção da cultura:** Disseminar os princípios e benefícios por toda a TI.
 - **Governança de TI:** Identificar obstáculos nos processos de controle e auditoria.
 - **Metas compartilhadas:** Focar no produto final em vez de metas individuais.
 - **Automatização prioritária:** Focar em testes e deploys para garantir escalabilidade na nuvem.

Tema 4 – Autenticação e Autorização

Garante o controle sobre quem acessa os recursos (autenticação) e quais ações podem executar (autorização).

- **Abordagem DevSecOps:** Diferente do modelo tradicional onde a segurança é a etapa final, no DevSecOps ela é uma responsabilidade compartilhada e integrada desde o planejamento.
- **Uso de IA (ChatGPT):** Scripts de IaC podem ser analisados por modelos de linguagem para encontrar falhas de segurança e sugerir conselhos de correção.
- **Amazon Inspector:** Serviço da AWS para avaliação automatizada e contínua de vulnerabilidades em instâncias EC2 e aplicações, gerando relatórios detalhados com recomendações. O Terraform pode ser usado para automatizar a criação de perfis e a execução dessas avaliações.

Tema 5 – Segurança como Código

Redefine a proteção ao integrar verificações de segurança diretamente nos pipelines de CI/CD e na infraestrutura (IaC).

- **Ferramentas de Avaliação:**
 - **SAST:** Teste de segurança estática no código-fonte.
 - **SCA:** Análise de composição de software e riscos de código aberto.
 - **IAST:** Teste interativo que monitora a aplicação em execução.
 - **DAST:** Teste dinâmico que simula ataques externos.
- **Práticas com AWS e Terraform:** Incluem o gerenciamento seguro de credenciais via IAM, uso de variáveis sensíveis, criptografia de arquivos de estado, revisões de código regulares e automação de patches via AWS Systems Manager